INTEGRAÇÃO DE SISTEMAS DE INFORMAÇÃO

Performance, Transferências e Estatísticas no Futebol Europeu

UPCA

Contents

1. Enquadramento 6

1.1. Identificação da Equipa de Desenvolvimento 6

1.2. Ecossistema Tecnológico Utilizado 7

1.3. Acesso ao Repositório do Projeto 7

2. Problema 8

2.1. Descrição do Cenário e Contexto de Negócio 8

2.2. Fontes de Dados e Heterogeneidade de Formatos ("E") 9

2.3. Desafios de Integração e Transformação de Dados ("T") 10

2.4. Persistência e Apoio à Tomada de Decisão ("L") 11

3. Estratégia Utilizada 12

3.1. Arquitetura Global e Ciclo de Vida dos Dados 12

3.2. **Modelo Relacional e Esquema de Dados (PostgreSQL)** 13

3.2.1. Camada de Staging: Tabela raw_api_standings 13

3.2.2. Camada Analítica (Data Mart): Tabela fact_club_performance 13

3.3. Mapeamento Comparativo de Operadores e Componentes 15

3.4. Critérios de Transformação e Lógica de Negócio Aplicada 16

4. Transformações 17

4.1. Pipeline de Transformação em KNIME Analytics Platform 17

4.1.1. Ramo 1: Ingestão e Preparação dos Dados da API (PostgreSQL) 18

4.1.2. Ramo 2: Processamento e Agregação de Jogadores (CSV) 18

4.1.3. Ramo 3: Extração e Mapeamento de Metadados (XML) 19

4.1.3.1. Fusão dos Dados e Lógica de Negócio ("T") 19

4.1.3.2. Persistência e Exportação (Fase 3 - "L") 20

4.2. Pipeline de Transformação em Apache Hop 20

4.3. Visualização e Suporte à Decisão no Microsoft Power BI Desktop 22

5. Jobs e Orquestração de Processos 23

5.1. Orquestração Sequencial do Ciclo de Integração 23

5.2. Implementação no Apache Hop (Workflow .hwf) 24

5.3. Orquestração e Controlo de Processo no KNIME 25

6. Vídeo com Demonstração 26

7. Conclusão e Trabalhos Futuros 27

7.1. Síntese dos Resultados e Objetivos Atingidos 27
7.2. Análise Comparativa: KNIME Analytics Platform vs. Apache Hop 28
7.3. Trabalhos Futuros 29
8. Referências Bibliográficas 30

Figura 1 - Diagrama panorâmico do Workflow completo no KNIME 17
Figura 2 - Diagrama panorâmico do Workflow completo no Apache Hop 20
Figura 3 - Dashboard Power BI 22
Figura 4 - Diagrama do Workflow de Orquestração .hwf no Apache Hop 24
Figura 5 - Diagrama de controlo de dependências e orquestração por portas de fluxo no KNIME 25
Figura 6 - Knime Video 26
Figura 7 - Apache Hop Video 26

Tabela 1 - Comparação de Componentes 15

1. Enquadramento

O presente relatório documenta o desenvolvimento do **Trabalho Prático I** no âmbito da unidade curricular de **Integração de Sistemas de Informação (ISI)**, integrada no plano curricular da **Licenciatura em Engenharia de Sistemas Informáticos (LESI)** da **Escola Superior de Tecnologia (EST)** da **Universidade Politécnico do Cávado e do Ave (UPCA)**, referente ao ano letivo de 2026/27.

A unidade curricular foca-se na interoperabilidade entre sistemas organizacionais e arquiteturas de integração de dados. O trabalho foi estruturado com o objetivo de consolidar os conceitos teóricos inerentes aos processos de **ETL (*Extract, Transformation and Load*)**, explorando o ciclo de vida completo do dado: desde a recolha em fontes heterogéneas e formatos não normalizados até à transformação lógica, persistência em bases de dados relacionais e exposição dos resultados em soluções analíticas para suporte à tomada de decisão.

1.1. Identificação da Equipa de Desenvolvimento

O projeto foi executado em regime de grupo por dois estudantes, cumprindo a diretriz de exploração individual de plataformas distintas ao nível do processamento principal:

- **Elemento 1:** Afonso da Costa Veloso — a28542
    - *Plataforma:* **KNIME Analytics Platform**
- **Elemento 2:** Rafael Croccia Pinto Grenha Silva — a28554
    - *Plataforma:* **Apache Hop**

1.2. Ecossistema Tecnológico Utilizado

Para além dos motores centrais de ETL desenvolvidos de forma independente por cada elemento, a infraestrutura partilhada de suporte assenta nas seguintes ferramentas e tecnologias:

- **PostgreSQL:** Sistema de gestão de bases de dados relacional responsável pelo armazenamento da camada de dados brutos (*Staging Layer* / ingestão de API) e da camada de persistência final (*Data Mart* analítico);
- **Microsoft Power BI Desktop:** Camada de visualização executiva (*Business Intelligence*), encarregue do consumo da tabela final consolidada via ligação nativa à base de dados para geração de relatórios interativos e acompanhamento de métricas-chave (KPIs);
- **Git & GitHub:** Plataforma de controlo de versões distribuído para partilha de esquemas SQL, ficheiros de configuração de metadados, *workflows* e artefactos produzidos durante o projeto.

1.3. Acesso ao Repositório do Projeto

Todos os recursos criados, incluindo ficheiros de fluxo do KNIME (.knwf), ficheiros de pipeline do Apache Hop (.hpl), ficheiro XML de metadados e o modelo de relatório .pbix do Power BI, encontram-se disponíveis no repositório oficial do projeto:

- **Repositório Git:** https://github.com/Costaa18/ISI_TP01_a28542_a28554

2. Problema

2.1. Descrição do Cenário e Contexto de Negócio

No contexto da indústria do desporto profissional contemporâneo, a gestão financeira e o recrutamento de atletas tornaram-se áreas altamente dependentes de abordagens analíticas (*Sports Analytics*). Um dos maiores desafios estratégicos que as administrações de clubes de futebol enfrentam prende-se com a **avaliação da eficiência do investimento**: compreender até que ponto o valor de mercado agregado de um plantel se traduz, de forma proporcional, em rendimento desportivo e pontos conquistados dentro de campo.

Contudo, esta análise depara-se sistematicamente com uma forte barreira técnica: a **heterogeneidade e dispersão das fontes de informação**. Os dados desportivos oficiais e as classificações são tipicamente disponibilizados em tempo real através de serviços web e APIs REST; os dados financeiros detalhados e as cotações históricas de mercado de cada jogador residem frequentemente em ficheiros tabulares consolidados; e os regulamentos competitivos, metas de qualificação europeia e parâmetros de Fair Play Financeiro encontram-se estruturados em formatos hierárquicos e normativos.

O objetivo deste projeto consiste em conceber, implementar e comparar processos de **ETL (*Extract, Transform, Load*)** capazes de integrar estas fontes díspares num modelo dimensional unificado, possibilitando aferir a rentabilidade do investimento na principal liga inglesa (*Premier League*).

2.2. Fontes de Dados e Heterogeneidade de Formatos ("E")

Para dar resposta a este desafio e satisfazer os requisitos de ingestão de origens e formatos diversos definidos no enunciado, foram mobilizadas três fontes heterogéneas:

1. **Web Service (Formato JSON):**
    - *Fonte:* API pública football-data.org.
    - *Conteúdo:* Tabela de classificação em tempo real da *Premier League*, incluindo número de jogos disputados, vitórias, empates, derrotas, golos marcados/sofridos e total de pontos somados.
    - *Natureza:* Consumo remoto via chamadas HTTP GET autenticadas através de cabeçalhos específicos (X-Auth-Token).
2. **Origem Tabular Estática (Formato CSV):**
    - *Fonte:* Dataset consolidado do *Transfermarkt* (disponibilizado via Kaggle).
    - *Conteúdo:* Registo individual de futebolistas, incluindo vínculo contratual ao respetivo clube, data de nascimento/idade, nacionalidade e valor de mercado estimado em euros (market_value_in_eur).
    - *Natureza:* Ficheiro local com grande volume de dados normalizados, exigindo operações de filtragem e agregações em lote.
3. **Origem Hierárquica de Metadados (Formato XML):**
    - *Fonte:* Ficheiro estruturado competicoes_referencia.xml, parametrizado com base em relatórios da UEFA e diretrizes regulamentares da liga.
    - *Conteúdo:* Metadados de contexto competitivo, tais como coeficientes europeus, limiares médios históricos de pontos para qualificação para a *UEFA Champions League* e *Europa League*, linhas de segurança contra a descida de divisão e benchmarks financeiros de custo esperado por ponto.
    - *Natureza:* Documento hierárquico processado através de linguagens de consulta estruturada (*XPath*).

2.3. Desafios de Integração e Transformação de Dados ("T")

A reconciliação destas três fontes coloca vários obstáculos práticos que a camada de transformação de negócio teve de resolver:

- **Inconsistência de Nomenclatura e Chaves de Junção:** Diferenças de grafia entre a designação oficial dos clubes retornada pela API e as convenções utilizadas nas tabelas de jogadores (ex.: sufixos como " FC", " AFC" ou espaços adicionais), exigindo a aplicação de **Expressões Regulares (Regex)** para normalização e composição de chaves comuns;
- **Granularidades Distintas de Dados:** A necessidade de converter dados a nível de atleta (micro) para métricas consolidadas a nível de clube (macro), através de operadores de agrupamento (*GroupBy*), cálculo de somas e médias aritméticas (idade média e montante global do plantel);
- **Tratamento de Anomalias e Nulos:** Existência de atletas sem cotação de mercado atribuída, obrigando a estratégias de imputação de valores padrão para não corromper os cálculos financeiros globais;
- **Cálculo de Indicadores de Decisão:** Formulação de novas variáveis analíticas capazes de responder à pergunta de negócio, nomeadamente o rácio de **Custo por Ponto** investido ($\frac{\text{Valor do Plantel}}{\text{Pontos}}$), a projeção matemática de pontuação para o término das 38 jornadas e a margem de segurança relativamente às metas europeias estabelecidas no XML.

2.4. Persistência e Apoio à Tomada de Decisão ("L")

O problema só se considera ultrapassado no momento em que a informação tratada é exposta aos intervenientes de negócio de forma acionável. Para tal, a arquitetura contempla a carga do modelo analítico final numa tabela de *Data Mart* no SGBD **PostgreSQL** (fact_club_performance) e a exportação em formato estruturado (CSV).

A partir deste repositório central, os dados alimentam um dashboard interativo no **Microsoft Power BI Desktop**, munido de gráficos de dispersão, indicadores-chave de desempenho (KPIs) e matrizes de classificação que permitem identificar de imediato casos de sobrevalorização desportiva (*overperforming*) ou défice de retorno financeiro (*underperforming*).

3. Estratégia Utilizada

3.1. Arquitetura Global e Ciclo de Vida dos Dados

A abordagem adotada assenta no paradigma clássico de **ETL (*Extract, Transform, Load*)**, decomposto em três fases sequenciais e interdependentes concebidas para garantir a integridade, rastreabilidade e interoperabilidade dos dados entre sistemas heterogéneos:

- **Fase 1:** Recolha e Preparação de Dados ("E"): Ingestão paralela de três fontes em formatos distintos (JSON dinâmico via API REST com autenticação por token, CSV local com centenas de atletas e XML hierárquico com metas competitivas). A API é descarregada e colocada numa tabela de *staging* relacional no **PostgreSQL** (raw_api_standings), assegurando a preservação do estado original dos dados brutos e prevenindo bloqueios por limites de chamadas (*rate limits*).
- **Fase 2:** Processamento e Transformação de Dados ("T"): Execução do processamento analítico nos respetivos motores (**KNIME** e **Apache Hop**). Esta fase inclui a normalização sintática de entidades através de Expressões Regulares (Regex), conversão de tipos de dados, agregação de dados individuais de atletas para nível de clube, múltiplos cruzamentos relacionais (*Joins/Lookups*) e derivação de variáveis matemáticas e lógicas de suporte à decisão.
- **Fase 3**: Persistência e Exportação ("L"): Carregamento atómico da tabela dimensional final (fact_club_performance) na base de dados PostgreSQL e exportação simultânea para um ficheiro consolidado .csv. A camada de dados resultante é consumida pelo **Power BI Desktop** para disponibilizar uma interface analítica interativa com indicadores e gráficos de correlação.

3.2. Modelo Relacional e Esquema de Dados (PostgreSQL)

Para suportar o fluxo entre a extração e a carga analítica, foram concebidas duas estruturas de dados no SGBD PostgreSQL:

3.2.1. Camada de Staging: Tabela raw_api_standings

Armazena a informação em bruto retornada pelo endpoint da API da *Premier League*, permitindo ao pipeline do motor de ETL ler dados persistidos de forma segura e determinística:

- id SERIAL PRIMARY KEY: Identificador único gerado automaticamente pelo SGBD.
- competition_code VARCHAR(10): Código da competição (PL).
- position INT, team_name VARCHAR(150), played_games INT, won INT, draw INT, lost INT: Métricas desportivas em formato numérico e textual.
- points INT, goals_for INT, goals_against INT, goal_difference INT: Parâmetros de pontuação e golos da classificação.
- ingested_at TIMESTAMP: Registo temporal de execução para auditoria da recolha.

3.2.2. Camada Analítica (Data Mart): Tabela fact_club_performance

Consolida os dados após o cruzamento dos três canais e o cálculo dos indicadores de negócio, servindo de base direta ao Power BI:

- id SERIAL PRIMARY KEY: Chave primária substituta da dimensão.
- competition_code, league_name, team_name, short_name: Identificadores descritivos e limpos dos clubes.
- position, played_games, won, draw, lost, points, goal_difference: Dados desportivos apurados.

- total_squad_value_eur NUMERIC(15, 2): Cotação agregada do plantel calculada via agregação do CSV.
- cost_per_point_eur NUMERIC(15, 2): Rácio matemático entre o valor de mercado e os pontos ganhos.
- projected_points NUMERIC(6, 2): Ritmo pontual extrapolado para os 38 jogos da época.
- sporting_outlook VARCHAR(50): Classificação de risco/objetivo calculada pelo motor de regras (UCL Pace, UEL Pace, Relegation Threat, Balanced).
- processed_at TIMESTAMP: Carimbo de data/hora da geração da tabela.

3.3. Mapeamento Comparativo de Operadores e Componentes

De modo a cumprir o objetivo de avaliação e experimentação de ferramentas distintas por cada elemento da equipa, a estratégia técnica garantiu a equivalência funcional entre os nós do **KNIME Analytics Platform** e os *transforms* do **Apache Hop**:

Tabela 1 - Comparação de Componentes

Etapa do Processo	***Operação Lógica***	***Componente em KNIME***	***Transform em Apache Hop***
Ingestão (Fase 1)	Chamada HTTP à API REST	GET Request	REST Client
	Parsing do formato JSON	JSON to Table / JSON Path	JSON Input
	Leitura de Dataset CSV	CSV Reader	CSV File Input
	Parsing de Metadados XML	XML Reader → XML to JSON → JSON Path	Get data from XML (via XPath)
	Leitura da Camada Staging	PostgreSQL Connector + DB Reader	Database Lookup / Table Input
Tratamento (Fase 2)	Normalização com Regex	String Manipulation	Replace in string
	Agrupamento e Métricas	GroupBy	GroupBy
	Tratamento de Nulos	Missing Value	If field value is null
	Junções Relacionais	Joiner	Stream Lookup
	Cálculos Aritméticos	Math Formula	Calculator
	Classificação Condicional	Rule Engine	Formula
	Seleção de Schema	Column Filter e Column Rename	Select values
Carga (Fase 3)	Escrita em Base de Dados	DB Insert	Table Output
	Geração de Exportação	CSV Writer	Text file output

3.4. Critérios de Transformação e Lógica de Negócio Aplicada

Para responder diretamente aos critérios de mais-valia destacados no enunciado da unidade curricular, foram desenhadas quatro regras de transformação fundamentais:

1. Higienização de Strings com Regex (Mais-valia obrigatória): Eliminação sistemática de ruído de nomenclatura nos nomes dos clubes através de expressões que expurgam sufixos como \s+(FC|AFC|F\.C\.)\s*$ e prefixos iniciais, aplicando funções de remoção de espaços em branco marginais (strip).
2. **Homogeneização de Granularidade:** O ficheiro players.csv possui uma granularidade atómica (um registo por futebolista). O operador de agrupamento condensa centenas de registos numa única linha por clube antes de qualquer tentativa de cruzamento, computando a massa salarial/patrimonial agregada.
3. **Métricas Derivadas de Retorno de Investimento (ROI):**

$$\text{cost_per_point_eur} = \frac{\text{total_squad_value_eur}}{\text{points}}$$

$$\text{projected_points} = \left(\frac{\text{points}}{\text{played_games}}\right) \times 38$$

4. **Segmentação Competitiva e Suporte à Decisão:** Avaliação da performance através da diferença entre os pontos projetados e as metas estipuladas no XML para qualificação europeia e manutenção, categorizando os emblemas desportivos em quatro níveis estratégicos (UCL Pace, UEL Pace, Mid-table Safety e Relegation Threat).

4. Transformações

Nesta secção descreve-se detalhadamente a lógica de processamento e manipulação de dados implementada nas duas ferramentas adotadas pela equipa, demonstrando a equivalência funcional e o tratamento rigoroso aplicado aos dados ao longo de todo o ciclo de vida.

4.1. Pipeline de Transformação em KNIME Analytics Platform

O fluxo desenvolvido no **KNIME** processa as três origens de dados de forma assíncrona e modular através de ramos independentes, convergindo numa sequência de cruzamentos relacionais e derivação analítica de métricas.

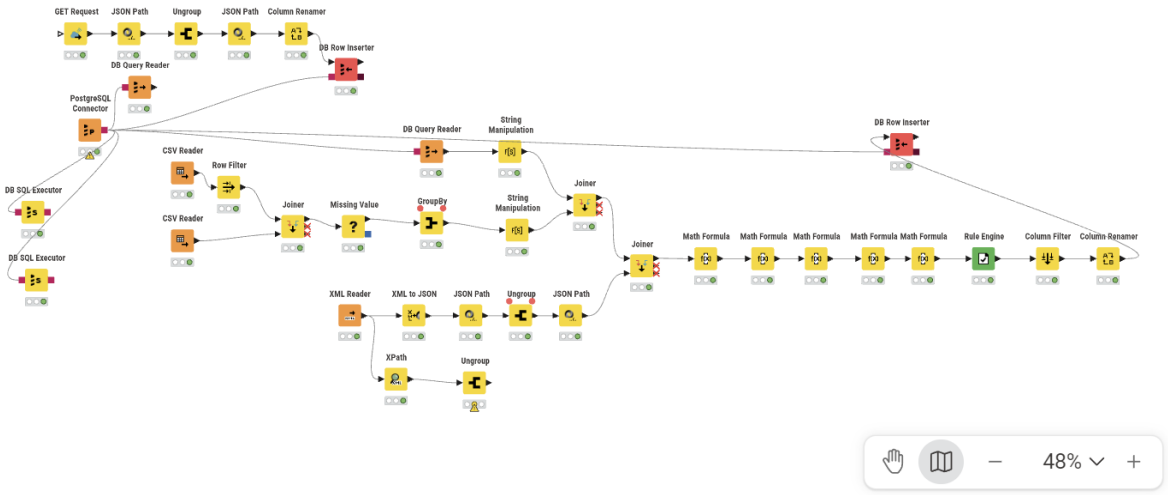


Figura 1 - Diagrama panorâmico do Workflow completo no KNIME

O encadeamento de operadores estrutura-se através das seguintes etapas lógicas:

4.1.1. Ramo 1: Ingestão e Preparação dos Dados da API (PostgreSQL)

- **Conector Relacional (PostgreSQL Connector):** Estabelece a ligação JDBC à base de dados local (localhost:5432), garantindo a comunicação transacional com a camada de *staging*.
- **Leitura da Camada de Staging (DB Query Reader / DB Reader):** Executa a instrução SELECT * FROM raw_api_standings;, importando para o espaço de memória do KNIME a tabela com os 20 clubes da *Premier League* recolhidos da API.
- **Higienização de Strings com Regex (String Manipulation):**

- *Objetivo:* Padronizar a grafia das equipas para viabilizar os cruzamentos relacionais (*joins*) com o CSV.
- *Expressão utilizada:*
    - strip(regexReplace(regexReplace($team_name$, "\\s+(FC|AFC|F\\.C\\.)\\s*$", ""), "^(AFC)\\s+", ""))
- *Efeito prático:* Remove terminações como " FC" ou " AFC" e expurga espaços marginais em branco, transformando designações como "Arsenal FC" em "Arsenal".

4.1.2. Ramo 2: Processamento e Agregação de Jogadores (CSV)

1. **Leitura de Atletas (CSV Reader):** Leitura do ficheiro players.csv, extraindo as colunas player_id, name, current_club_id, market_value_in_eur e datas.
2. **Cruzamento com Clubes (Joiner):** Faz a junção com o catálogo clubs.csv (filtrado para a *Premier League*) através da chave current_club_id == club_id.
3. **Tratamento de Nulos (Missing Value):** Substitui células nulas (?) na cotação de mercado por um valor numérico fixo 0 para não corromper cálculos aditivos posteriores.
4. **Agregação Estrutural (GroupBy):**
    - *Chave de Agrupamento:* Nome do clube (team_name).
    - *Operadores Aplicados:*
        - Sum(market_value_in_eur) → computa o valor global do plantel (total_squad_value_eur);
        - Count(player_id) → contabiliza o tamanho do plantel.
    - *Resultado:* Redução de centenas de linhas de jogadores para exatamente uma linha condensada por cada clube.
5. **Higienização do Nome do Clube (String Manipulation):** Aplicação da mesma expressão regular utilizada no Ramo 1, garantindo simetria de chaves de texto.

4.1.3. Ramo 3: Extração e Mapeamento de Metadados (XML)

1. **Leitura Hierárquica (XML Reader):** Carregamento do documento normativo competicoes_referencia.xml.
2. **Serialização e Desdobramento (XML to JSON → JSON Path → Ungroup):**
   - Conversão da árvore XML em estrutura JSON.
   - nó JSON Path isola a coleção $.sports_governance.competitions.competition[*].
   - O nó Ungroup desdobra a coleção em linhas físicas distintas (uma linha por competição: PL e PPL).
3. **Extração Final de Metadados (JSON Path):** Extrai as variáveis numéricas e textuais para colunas convencionais: competition_code, target_champions_league, target_europa_league, target_relegation_safety e expected_cost_per_point.

4.1.3.1. Fusão dos Dados e Lógica de Negócio ("T")

1. **Primeira Junção Relacional (Joiner):** Faz o cruzamento entre o Ramo 1 (API/PostgreSQL) e o Ramo 2 (CSV Agrupado) com base no nome do clube limpo (Inner Join), associando o desempenho desportivo ao investimento financeiro.
2. **Segunda Junção de Metadados (Joiner):** Cruza a tabela unificada anterior com o Ramo 3 (XML) através do identificador competition_code == competition_code (PL), enriquecendo cada clube com as metas da UEFA.
3. **Cálculos Aritméticos de Rentabilidade (Math Formula):**

- *Custo por Ponto Conquistado:*

$$\text{cost_per_point_eur} = \frac{\text{total_squad_value_eur}}{\text{points}}$$

- *Projeção Pontual a 38 Jogos:*

$$\text{projected_points} = \left(\frac{\text{points}}{\text{played_games}}\right) \times 38$$

4. **Categorização Estratégica (Rule Engine):** Avalia a posição e ritmo projetado relativamente aos limiares do XML, derivando a coluna de negócio sporting_outlook:

$projected_points$ >= $target_champions_league$ => "UCL Pace"

$projected_points$ >= $target_europa_league$ => "UEL Pace"

$projected_points$ < $target_relegation_safety$ => "Relegation Threat"

TRUE => "Mid-table Safety"

5. **Filtragem e Limpeza de Schema (Column Filter & Column Rename):** Expurgam-se colunas técnicas residuais e IDs duplicados, deixando as 16 colunas finais limpas prontas para escrita.

4.1.3.2. Persistência e Exportação (Fase 3 - "L")

- **Escrita em Base de Dados (DB Insert):** Insere os registos limpos na tabela analítica fact_club_performance no PostgreSQL.
- **Exportação Ficheiro Local (CSV Writer):** Gera em simultâneo o ficheiro fact_club_performance.csv no repositório local do projeto.

4.2. Pipeline de Transformação em Apache Hop

No ambiente do **Apache Hop**, a mesma arquitetura de negócio foi implementada tirando partido do paradigma de metadados e execução concorrente de *transforms* integrados num único pipeline modular

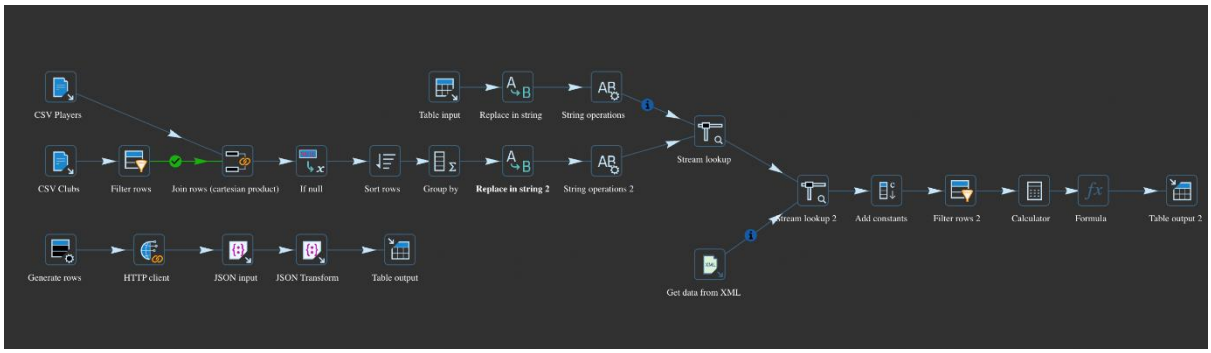


Figura 2 - Diagrama panorâmico do Workflow completo no Apache Hop

1. **Ingestão Remota da API:** Recorre ao transform **REST Client** com passagem dinâmica do cabeçalho X-Auth-Token. O payload retornado é decomposto pelo transform **JSON Input**, que mapeia as propriedades dos clubes em campos de

stream, seguindo para um **Table Output** responsável pela carga na tabela de staging do PostgreSQL.

2. **Leitura e Tratamento do Dataset CSV:** O transform **CSV File Input** lê os futebolistas locais. De seguida, aplica-se o transform **Replace in string** com suporte a Expressões Regulares (Regex Evaluation) para expurgar sufixos de clubes, seguido de um **If field value is null** para imputar zeros nos valores de mercado.
3. **Agregação:** O transform **Memory Group By** agrega a cotação somada por emblema desportivo.
4. **Parsing do XML de Referência:** Utiliza-se o transform **Get data from XML**, configurado com consultas XPath diretas aos nós <competition> e elementos filhos para disponibilizar os parâmetros de qualificação.
5. **Cruzamentos Relacionais:** A fusão das três fontes é orquestrada através de passos sequenciais de **Merge Join** (ou **Stream Lookup**) usando ordenação prévia com **Sort rows** para assegurar junções sem perda de integridade.
6. **Métricas e Segmentação:** O transform **Calculator** executa as divisões e extrapolações temporais de pontos, enquanto um transform **Switch / Case** (ou **Formula**) atribui a categoria sporting_outlook.
7. **Carga e Persistência:** Os dados consolidados são gravados via transform **Table Output** no PostgreSQL (fact_club_performance) e exportados para ficheiro através de **Text file output**

4.3. Visualização e Suporte à Decisão no Microsoft Power BI Desktop

A camada de visualização consome diretamente a tabela fact_club_performance através de ligação nativa à base de dados PostgreSQL, organizando a informação num dashboard executivo de página única:



Figura 3 - Dashboard Power BI

O relatório foi concebido com os seguintes componentes-chave:

- **Segmentação Estratégica (Slicer):** Permite filtrar os visuais em tempo real por objetivo desportivo (sporting_outlook), isolando clubes com aspirações de Champions League ou sob risco de despromoção.
- **Painel Superior de Indicadores (KPI Cards):**
    - *Valor Total de Mercado da Liga:* Montante global apurado;
    - *Custo Médio por Ponto:* Mede o investimento médio exigido por cada ponto conquistado;
    - *Média Pontual da Competição:* Ritmo médio da tabela classificativa;

- **Gráfico de Dispersão Central:**
    - Eixo X: total_squad_value_eur (Investimento financeiro).
    - Eixo Y: points (Pontos conquistados).
    - Legenda: sporting_outlook.
    - Tamanho das Bolhas: goals_for.

5. Jobs e Orquestração de Processos

Enquanto as pipelines e transformações documentadas no capítulo anterior se dedicam à manipulação granular e fluxo de registos de dados em memória (*data-flow*), a camada de **Jobs / Workflows de Controlo (*process-control*)** assegura a orquestração global, gestão de dependências sequenciais, controlo transacional e tratamento de exceções.

Neste projeto, a orquestração do processo de integração garante que as três fases de dados ocorrem de forma determinística, impedindo inconsistências de dados ou tentativas de cruzamento analítico antes da conclusão bem-sucedida das cargas iniciais.

5.1. Orquestração Sequencial do Ciclo de Integração

O processo global de controlo segue a seguinte máquina de estados lógica:

1. **Validação de Conectividade e Infraestrutura:** Verificação prévia da disponibilidade do serviço de base de dados PostgreSQL (localhost:5432) e da acessibilidade do ficheiro normativo XML e dataset CSV.
2. **Execução do Job de Ingestão e Staging (Fase 1):** Disparo da chamada à API externa (football-data.org), receção do payload JSON e persistência isolada na tabela raw_api_standings. A progressão do fluxo fica condicionada à confirmação de escrita transacional nesta etapa.
3. **Disparo do Motor de Processamento Principal (Fase 2):** Arranque da pipeline de integração que executa a limpeza com Regex, agregações, múltiplos

cruzamentos (*Joins*) e aplicação das regras de negócio de custo por ponto e metas da UEFA.

4. **Carga Analítica e Auditoria (Fase 3):** Gravação atómica dos registos apurados na tabela fact_club_performance e exportação simultânea do ficheiro estruturado .csv.
5. **Geração de Logs e Tratamento de Erros:** Registo temporal e dimensional do número de registos processados e captura de eventuais exceções de integridade referencial ou falhas de rede.

5.2. Implementação no Apache Hop (Workflow .hwf)

No **Apache Hop**, esta arquitetura foi implementada através de um ficheiro de *Workflow* nativo (.hwf), separando claramente a lógica de controlo da lógica de transformação:

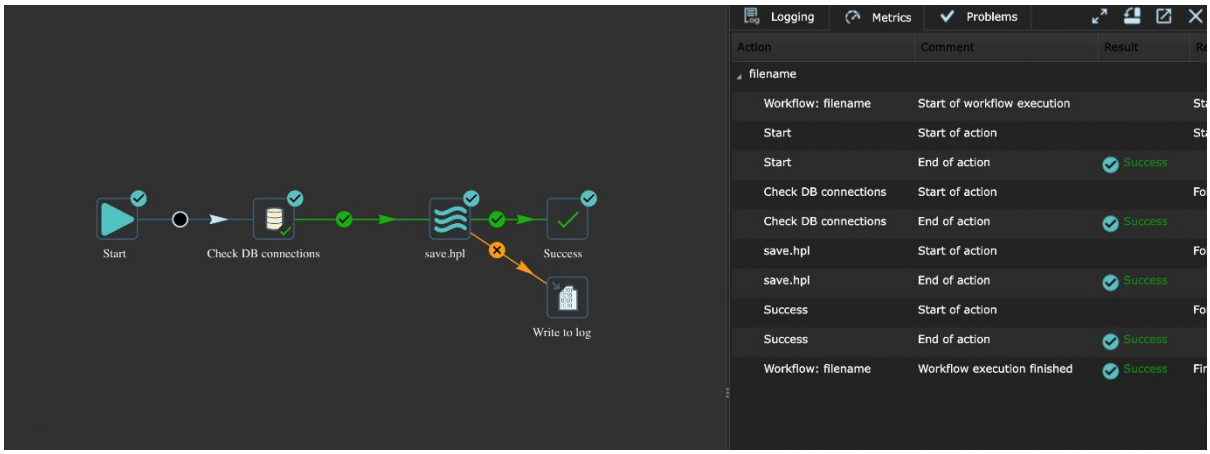


Figura 4 - Diagrama do Workflow de Orquestração .hwf no Apache Hop

- **Nó START:** Ponto de entrada do agendamento ou execução manual;
- **Action Check DB connections:** Testa a conectividade com a base de dados PostgreSQL antes de qualquer operação de I/O;
- **Action Pipeline (Ingestão API):** Executa a pipeline de recolha de dados da classificação;
- **Avaliação de Sucesso / Avaliação Condicional:** Em caso de sucesso (True), caso ocorra falha de conectividade ou resposta de erro HTTP, o fluxo desvia para uma ação de registo de erro (Write to log);
- **Action Success:** Conclusão formal do ciclo de processamento

5.3. Orquestração e Controlo de Processo no KNIME

No ambiente do **KNIME Analytics Platform**, o controlo de dependências foi assegurado através do encapsulamento modular em **Componentes / Metanós** e da utilização de variáveis de fluxo (*Flow Variables*):

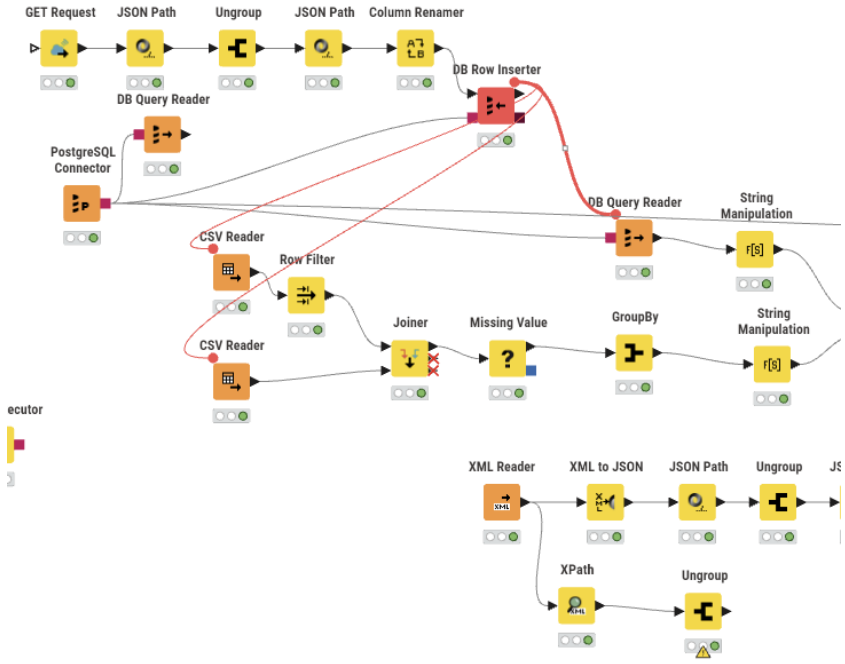


Figura 5 - Diagrama de controlo de dependências e orquestração por portas de fluxo no KNIME

- **Ativação dos Portos de Controlo:** Foram expostas as portas de controlo do nó DB Row Inserter (porta de saída no canto superior direito) e dos nós de arranque do ramo analítico (portas de entrada no canto superior esquerdo).
- **Propagação de Estado:** Ao ligar a porta vermelha de saída do DB Row Inserter à entrada dos nós de leitura (DB Query Reader, CSV Reader e XML Reader), é estabelecida uma dependência sequencial direta.
- **Bloqueio e Desbloqueio:** O KNIME mantém os nós de leitura em estado de espera (queued/idle) até que o nó DB Row Inserter mude para o estado executado com sucesso (verde). Apenas após a confirmação de escrita na base de dados é transmitido o gatilho (trigger) de execução, permitindo que a recolha e processamento dos dados subsequentes se iniciem com total garantia de coerência.

6. Vídeo com Demonstração

Como complemento à documentação escrita e para validação da operacionalidade dos artefactos desenvolvidos, foi gravado um vídeo demonstrativo com a execução integral dos pipelines e a respetiva atualização na camada de suporte à decisão.



Figura 6 - Knime Video



Figura 7 - Apache Hop Video

7. Conclusão e Trabalhos Futuros

7.1. Síntese dos Resultados e Objetivos Atingidos

O desenvolvimento do presente trabalho prático permitiu consolidar de forma integral os conceitos fundamentais associados à **Integração de Sistemas de Informação ao nível dos dados**, cumprindo com sucesso todas as etapas do ciclo de vida de processos de **ETL (*Extract, Transform, Load*)**:

1. **Heterogeneidade de Formatos e Fontes (Fase 1):** Demonstrou-se a capacidade de ingerir, descodificar e preparar dados oriundos de três ecossistemas distintos — chamadas dinâmicas a uma API REST com respostas em formato JSON, ficheiros tabulares normalizados em CSV e documentos hierárquicos normativos em formato XML;
2. **Processamento e Transformação de Negócio (Fase 2):** Foram aplicadas com sucesso técnicas de higienização sintática com recurso a **Expressões Regulares (Regex)** para resolução de assimetrias textuais e composição de chaves, agregações estatísticas por entidade desportiva (*GroupBy*) e múltiplos cruzamentos relacionais (*Joins/Lookups*);
3. **Persistência, Orquestração e Exposição (Fase 3):** Implementou-se uma arquitetura em duas camadas no SGBD **PostgreSQL** (*Staging Layer* e *Data Mart* analítico), suportada pela geração em simultâneo de ficheiros estruturados de exportação e visualização em tempo real num dashboard interativo em **Microsoft Power BI**.

Do ponto de vista da análise desportiva e financeira da *Premier League*, os dados integrados evidenciaram que **o volume financeiro investido num plantel não garante uma correlação linear estrita com a pontuação alcançada**. Através da métrica derivada de **Custo por Ponto** (cost_per_point_eur) e do gráfico de dispersão (*Scatter Plot*), foi possível identificar com clareza emblemas com elevadíssimo retorno de investimento (alta pontuação com massa salarial/patrimonial contida) e clubes com forte défice de rendimento face ao capital avaliado.

7.2. Análise Comparativa: KNIME Analytics Platform vs. Apache Hop

A exploração paralela de duas plataformas distintas por cada um dos membros do grupo permitiu estabelecer um balanço crítico entre paradigmas de desenvolvimento:

- **KNIME Analytics Platform:**
    - *Vantagens:* Interface gráfica intuitiva, rica em representações visuais imediatas dos dados em qualquer ponto da pipeline através de tabelas intermédias integradas; excelente suporte a manipulações estatísticas, cálculos matemáticos complexos (*Math Formula*) e gestão de tipos de dados;
    - *Limitações:* Consumo de memória RAM superior ao trabalhar com estruturas hierárquicas profundas de XML, exigindo a conversão intermédia para JSON (XML to JSON) para facilitar operações com caminhos XPath e desdobramentos de coleções.
- **Apache Hop:**
    - *Vantagens:* Motor extremamente leve, modular e desacoplado, com excelente desempenho multiplataforma em ambientes macOS (Apple Silicon); separação conceptual formal e rigorosa entre pipelines de dados (.hpl) e workflows de orquestração/controlo de execução (.hwf); transform de leitura nativa de XML (Get data from XML) muito flexível e performante com XPath direto;
    - *Limitações:* Curva de aprendizagem inicial ligeiramente mais acentuada no mapeamento de streams e maior rigor na exigência de passos de ordenação prévia de linhas (*Sort rows*) antes de operações de junção (*Merge Join*).

Em suma, ambas as ferramentas provaram ser robustas e plenamente capazes de responder aos requisitos de engenharia de dados estipulados no enunciado.

7.3. Trabalhos Futuros

Como perspetivas de evolução e expansão da infraestrutura concebida, identificam-se as seguintes oportunidades técnicas:

1. **Expansão Multiliga Automatizada:** Alargar a ingestão a outras competições europeias de topo (como a Liga Portugal Betclic, La Liga e Serie A) tirando partido do nó parametrizado no XML (competition_code), permitindo uma análise comparativa transfronteiriça da inflação do custo por ponto no futebol europeu;
2. **Ingestão Contínua e Streaming (Event-Driven):** Integrar serviços baseados em eventos com **Node-RED** ou brokers **MQTT (Mosquitto)** para reagir em tempo real ao término de cada partida, atualizando automaticamente as tabelas de staging sem necessidade de agendamento em lote (*batch*);
3. **Mecanismos Automáticos de Alerta e Notificação:** Implementar módulos de notificação por e-mail (via SMTP) ou *webhooks* para sistemas de mensagens (ex.: Discord/Teams) sempre que um clube transite de estado competitivo (ex.: entrada em risco iminente de despromoção Relegation Threat ou desvio crítico de Fair Play Financeiro);
4. **Armazenamento Não Relacional (Data Lake):** Incorporar bases de dados NoSQL (ex.: MongoDB) para persistir o histórico de payloads brutos em JSON da API para auditoria retrospetiva antes de qualquer limpeza.

8. Referências Bibliográficas

[1] **KNIME AG**, *KNIME Analytics Platform User Guide & Documentation*, 2026. [Online]. Disponível em: https://docs.knime.com/ [Acedido em: Outubro de 2026].

[2] **Apache Software Foundation**, *Apache Hop Documentation: Pipelines, Transforms & Workflows*, 2026. [Online]. Disponível em: https://hop.apache.org/manual/latest/ [Acedido em: Outubro de 2026].

[3] **Football-Data.org**, *RESTful API for Football Data - Documentation & Endpoints*, v4, 2026. [Online]. Disponível em: https://www.football-data.org/documentation/api [Acedido em: Outubro de 2026].

[4] **Cariboo, D.**, *Football Data from Transfermarkt: Players, Clubs and Valuations*, Kaggle Dataset, 2024. [Online]. Disponível em: https://www.kaggle.com/datasets/davidcariboo/player-scores [Acedido em: Outubro de 2026].

[5] **The PostgreSQL Global Development Group**, *PostgreSQL 16/17 Documentation: Data Definition and SQL Commands*, 2026. [Online]. Disponível em: https://www.postgresql.org/docs/ [Acedido em: Outubro de 2026].

[6] **Microsoft Corporation**, *Documentação do Power BI: Conceitos Básicos de Modelação e Visualização de Dados*, Microsoft Learn, 2026. [Online]. Disponível em: https://learn.microsoft.com/pt-pt/power-bi/ [Acedido em: Outubro de 2026].

[7] **W3C (World Wide Web Consortium)**, *XML Path Language (XPath) 3.1 & Extensible Markup Language (XML) 1.0*, W3C Recommendations. [Online]. Disponível em: https://www.w3.org/TR/xpath/ [Acedido em: Outubro de 2026].

[8] **Ferreira, L. & Ribeiro, Ó.**, *Enunciado do Trabalho Prático I: Processos de ETL e Integração de Sistemas de Informação ao Nível dos Dados*, Unidade Curricular de Integração de Sistemas de Informação (ISI), Licenciatura em Engenharia de Sistemas Informáticos (LESI), Escola Superior de Tecnologia (EST) – IPCA, Ano Letivo 2026/27.